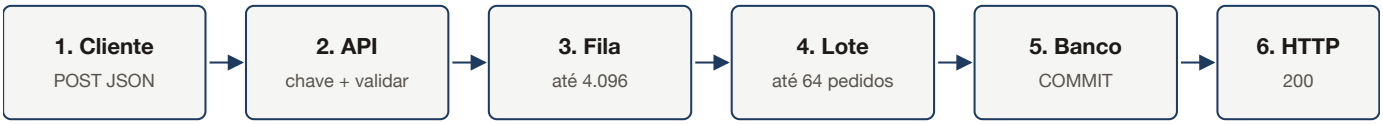
Do cliente até o banco

Passo a passo de um POST /transfers local neste autorizador. Duas APIs (:8080 e :8081) fazem o mesmo caminho; as duas gravam no mesmo PostgreSQL.

Visão geral

O cliente manda um JSON com um ID que ele escolhe e guarda. A API não aprova na hora. Ela coloca o pedido numa fila, junta vários, grava tudo numa transação e **só então** responde. Se a resposta se perder, o cliente manda o mesmo ID de novo e recebe a decisão já salva, sem mover o dinheiro outra vez.

A resposta 200 só sai depois do passo 5. Nada é “aprovado” só na memória.



Seis estágios. O HTTP 200 é o último, depois do COMMIT.

O que o cliente envia

URL: `POST http://127.0.0.1:8080/transfers` (ou :8081). Cabeçalho `Authorization: Bearer …` e corpo:

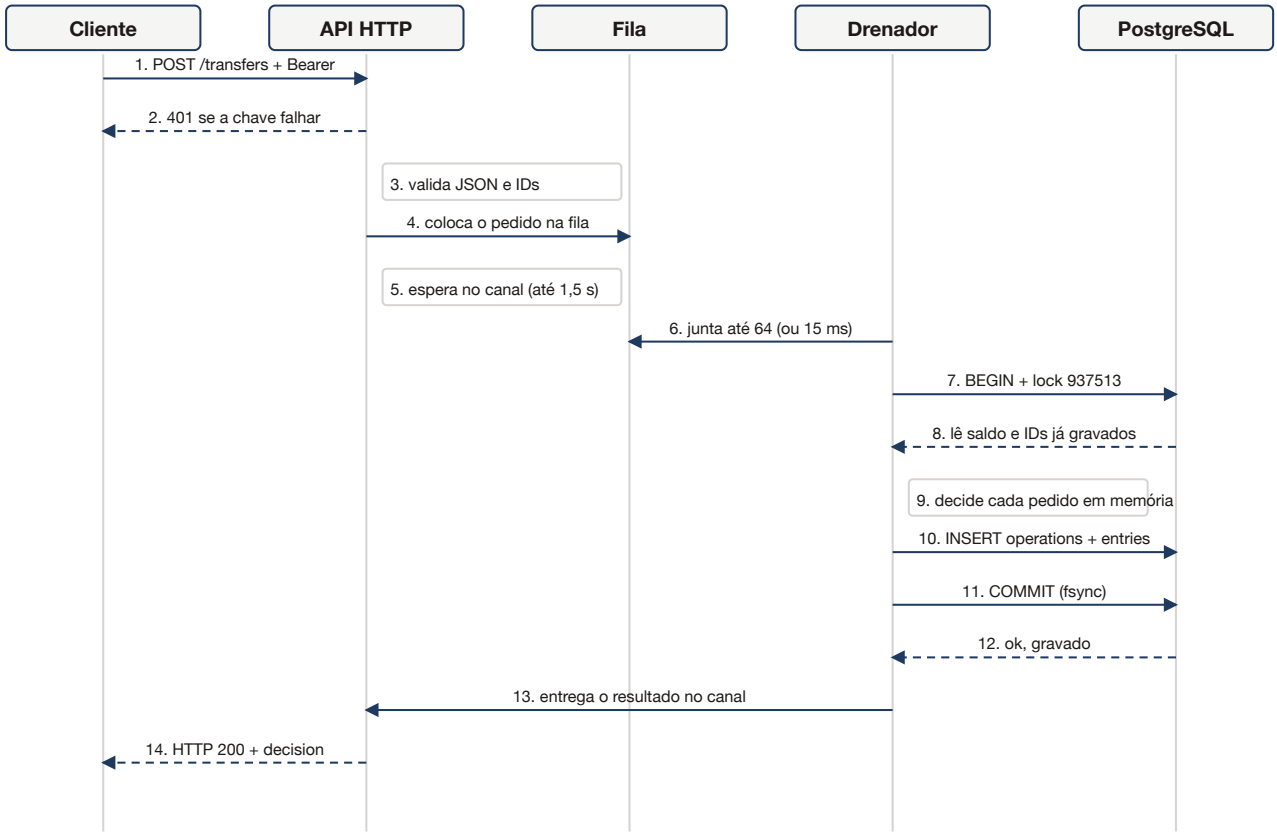
```
{
  "transaction_id": "abc-1",
  "source_id": 2,
  "destination_id": 1,
  "amount_minor": 1
}
```

Valores em centavos. transaction_id: letras, números, _ e -, até 64 caracteres. Origem e destino são contas locais positivas e diferentes. Destino externo usa outro campo e não entra neste passeio.

Diagrama de sequência

Setas cheias vão para a frente. Setas tracejadas voltam. O “Drenador” é um goroutine só, por API: ele pega a fila e fala com o banco.

Diagrama de sequência — transferência local



UML de sequência do caminho feliz. Códigos de erro saem mais cedo (tabela no fim).

Passo a passo

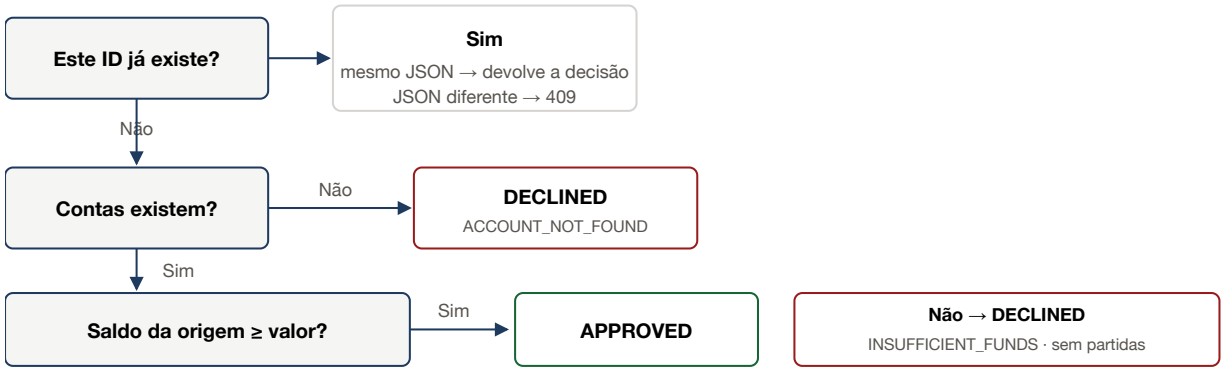
Na porta HTTP (ainda sem banco)

1. **Chega o POST.** O servidor Go aceita a conexão (timeout de leitura/escrita 3 s).
2. **Confere a chave.** O Bearer tem de ser exatamente a API_KEY. Se não for, responde 401 e para. /healthz e /readyz passam sem chave.
3. **Marca 1,5 s no relógio.** Esse prazo vale para o pedido inteiro. Estourar vira erro técnico (503), nunca DECLINED.
4. **Lê o JSON.** No máximo 4 KB, sem campos extras. Valida ID, contas e valor positivo. Origem igual ao destino é inválido. Falha → 400 INVALID_REQUEST.
5. **Enfileira.** O pedido entra num canal em memória, com até 4.096 lugares. A goroutine do HTTP não grava SQL. Ela espera um recado num canalzinho só daquele pedido.
6. **Fila cheia?** Responde na hora 503 e pede para repetir o mesmo ID. Nada foi ao banco.

O drenador junta um lote

7. **Pega o primeiro da fila** e tenta encher até 64. Se a fila já está cheia, drena os 64 na hora. Se está vazia, espera no máximo 15 ms (BATCH_WINDOW_MS) por mais gente.
8. **Abre uma transação SQL** com prazo próprio de 1,2 s. Se o cliente desistir no meio, os outros do lote continuam: o COMMIT não é cancelado por um timeout HTTP.
9. **Pede o lock 937513.** Só um lote no mundo (as duas APIs) grava de cada vez. A outra espera.
10. **Lê o saldo vigente** da origem e do destino: não é a coluna accounts neste caminho, é a última linha de entries daquela conta (se não houver partida, usa o saldo inicial).
11. **Lê operations** com aqueles transaction_id. Se o ID já está lá, compara o JSON. Igual → devolve a decisão antiga. Diferente → conflito 409, sem mexer no resto do lote.

Dentro do lote: como cada pedido vira APPROVED ou DECLINED

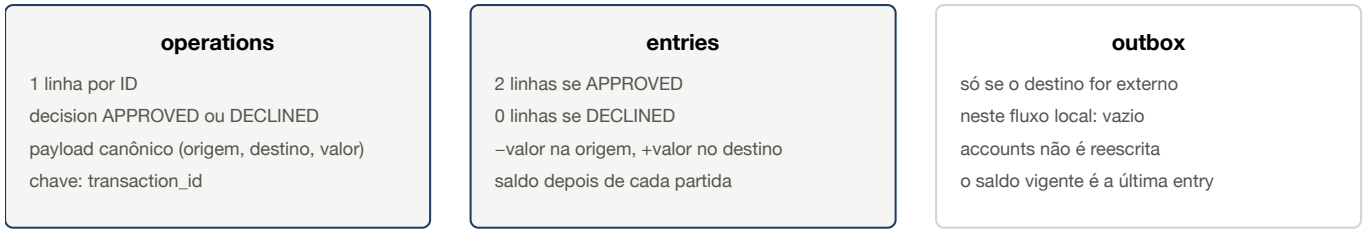


ledger.Decide: valor positivo, origem com saldo suficiente (igual pode), destino sem estourar o inteiro máximo.

Gravação no PostgreSQL

12. **Monta as partidas em memória.** Aprovação: −valor na origem e +valor no destino, cada uma com a próxima sequência e o saldo depois da linha. Negativa: zero partidas, mas a operação DECLINED ainda é gravada (aquele ID fica terminal).
13. **INSERT em operations.** Se outro processo gravou o mesmo ID no meio, o ON CONFLICT ignora; o lote relê e tenta de novo (até 3 vezes).
14. **INSERT em entries** das partidas aprovadas.
15. **INSERT na outbox** só se settlement for PENDING (destino externo). Transferência local não entra aqui.
16. **COMMIT.** Aí o disco confirma (fsync ligado). Antes disso não existe APPROVED para o cliente.
17. **Entrega no canal.** Cada HTTP parado acorda, monta o JSON e responde 200.

O que o COMMIT grava (aprovação local)



Três tabelas. A linha de accounts não é atualizada neste caminho; o extrato e o saldo leem a última partida.

Exemplo numérico

Conta 2 manda 1 centavo para a conta 1. Saldo da 2 era 1.000.000.000.000.

Tabela	O que fica
operations	id abc-1, APPROVED, INTERNAL, payload com origem 2, destino 1, amount 1
entries (origem)	conta 2, delta −1, sequence N+1, balance_after = saldo anterior − 1
entries (destino)	conta 1, delta +1, sequence M+1, balance_after = saldo anterior + 1
Resposta HTTP	`{"transaction_id":"abc-1","decision":"APPROVED","settlement_status":"INTERNAL"}`

O que o cliente pode receber

HTTP	Quando	O que fazer
200 APPROVED	COMMIT feito, duas partidas	Pronto
200 DECLINED	COMMIT feito, zero partidas (saldo, conta, overflow)	Aquele ID morreu; novo dinheiro pede novo ID
400	JSON ou regra de formato	Corrigir o pedido
401	Chave errada	Corrigir o Bearer
409	Mesmo ID, outro JSON	Não reutilizar o ID
503	Fila cheia, timeout, banco lento, COMMIT incerto	Repetir o *mesmo* ID. 503 nunca é negativa de negócio

Fila não é banco. Se a API morrer depois de enfileirar e antes do COMMIT, o PostgreSQL desfaz a transação. O cliente não recebeu 200: manda o mesmo ID. Se morrer depois do COMMIT e antes da resposta, o dinheiro já moveu: o reenvio só relê operations.

Este PDF descreve só a transferência local. GET /accounts/{id}/balance lê a última partida. Destino externo debita a origem, credita uma conta técnica (−1 a −32), grava outbox e um worker entrega no simulador. Consulta e Pix externo não aparecem no diagrama de sequência acima.